Binary Search :-

start = 0, end = n-1;

$\rightarrow$ (mid = $\frac{start + end}{2}$)

ceil value

$\frac{a}{b} + 1$

$\left[\frac{a+b-1}{b}\right]$

$\frac{28}{5}$

$\frac{28+5-1}{5} = \frac{32}{5} = 6$

Ques: Find square root of a given number.

(return floor value)

$int\left(\frac{a}{b}\right) \rightarrow$ floor value

$int\left(\frac{28}{5}\right) \rightarrow 5 \ldots\ldots \rightarrow 5$ only?

$\frac{25}{5} = \frac{25+5-1}{5}$

$= \frac{29}{5} = 5$

n = 25, ans = 5

n = 37, ans = 6

$\sqrt[2]{25}$

1x1

2x2

3x3

4x4

5x5

6x6

7x7

```
int ans=1;
for(int i=1; i<=n; i++) {
  if(i*i <= n)
    ans = i;
  else
    break;
}
return ans;
```

n = 1

ॐ

1 2 3 4 5 6 7 8 9 10 - - - - - 37

✓ ✓ ✓ ✓ ✓ ✓ × × × × × × ×

start

end

ans = ~~4~~ 6

S-I:- 1 19 37

start mid end

S-II: 1 9 ~~18~~

start ~~mid~~ end

S-III: ~~1~~ 4 8

~~start~~ end

S-IV: ~~start 5~~ 6 end

mid 8

S-V:- start mid ~~end~~

7 7 8

S-VI:- end start

6 7

```
int start=1, end=n, int ans=1;

while(start<=end){                    return ans/end;

mid=0   int mid = (start+end)/2;

if(mid x mid > n){
    end=mid-1;
}
else {
    start=mid+1;
    ans=mid;
}
}
```

piles = [3 6 7 11]

7/6

3/6 = 0 → 1 + 1 + 2 + 2

h = 8

→ not possible

→ possible

banana

1	2	3	4	5	6	7	8	9	10	11
3hr	2hr	1hr	1hr	1hr	1hr	1hr				1hr
6hr	3hr	2hr	2hr	2hr	1hr	1hr				1hr
7hr	4hr	3hr	2hr	2hr	2hr	1hr				1hr
11hr	5hr	4hr	3hr	3hr	2hr	2hr				1hr
			↓	↓	↓	↓				↓
			8hr	8hr	6hr	5hr				4hr

itt wanna

fun(c <= 8)

× × × ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

1 2 3 4 5 6 7 8 9 10 11

start end ~~mid~~ end

1 3 5

start mid end

4 5

start end

mid

3 4

end start

ans = ~~6~~ 4

3, 6, 7, 11

ceil

$\frac{7}{6}$

2

1 1

$\frac{piles[i]}{mid}$

$\frac{a}{b} = \frac{a+b-1}{b}$

$\frac{pile + mid - 1}{mid}$

3,6,7,11

```
int start = 1;   int ans=-1;
int end = maxPile(piles);

while( start <= end) {
    int mid = (start+end)/2;

    long totalHr = 0;
    for(int pile : piles) {
        totalHr += (pile+mid-1)/mid;
    }

    if(totalHr <= h) {
        ans = totalHr;
        end = mid-1;
    }
    else
        start = mid+1;
}

return start / ans;
```

2^{31} 2^{31}